

Instructions from Media.net:

1. This is a programming test.
2. The code area contains the function signature. Just complete the function alone. We will take care of the main function, headers, etc
3. Use 'Test' to run your solution against the sample test cases before submitting it.
4. Use 'Test with custom input' to run your solution against your custom test cases.
5. We run a **proprietary plagiarism check on each submission**. Please refrain from submitting code that is not written by you.

-
- *This test will be active from 31 Oct 2022 9:00 PM IST to 31 Oct 2022 10:30 PM IST.*
 - *You are free to choose any language from Python (python-2.7), C (gcc-4.8), C# (mono 6.12), Bash (bash 4.0.35), PHP (php 5.5.9), JavaScript (ES6), Scala (scala-2.11.4), MySQL (MySQL 5.6), Swift (5.5), Objective-C (clang 3.3), C++17 (gcc-9.2), Java7 (open-jdk-1.7.0), Text (text 6.10), MySQL (MySQL 8.0), GO (1.17.4), Python 3 (python-3.5), R (R-3.5.1), Java 8 (array support), Java 8 (oracle-jdk-1.8), Ruby 2 (ruby-2.0), Kotlin (openjdk8), Elixir (Erlang/OTP 24).*

Please refrain from performing the following restricted events:



S1



Q 1

Q 2

Q 3

All Sections & Questions

S1 Default Section (3 Questions)

#	QUESTION	TYPE	STATUS	ACTION
1	Fruits on Tree	</> Code	Not Attempted	Solve
2	Damaged Roads	</> Code	Not Attempted	Solve
3	Omega Primes	</> Code	Not Attempted	Solve

I am done solving all questions

[End Test](#)



S1



Q 1

Q 2

Q 3

S1 Default Section



Q1 >



Q2 >



Q3



Fruits on Tree

[Flag Question](#)

Problem Description

Given a **complete** rooted tree with **N** nodes numbered 1 to N. This tree has its leaves at the top and root at the bottom.

A **complete tree** is a tree which has all its **leaf at the same level**.

There are some fruits on every **leaf** of the tree and in order to get all the fruits you have to shake the tree any number of times.

But this tree is a little different than the rest it has following properties:

- Every node has its **capacity** value that represents the number of fruits a node can hold at any moment.
- Only one fruit Falls from **each** node to its parent node in one Shake.
- If a number of fruits at a node is **more** than its Capacity then excess fruits(greater than Capacity) on that node at that instant will fall to the ground. This process happens instantaneously hence **no shake** required.

The tree is rooted at 1. You may assume that root is one level above ground so all fruits which fall from root lands on the ground.

You have to find the **minimum** number of shakes you have to perform such that all the fruits are on the ground.

Problem Constraints

$$1 \leq N \leq 10^5$$

$$1 \leq A[i], B[i] \leq 10^9$$

$$1 \leq C[i][0], C[i][1] \leq N$$

$$A[i] = 0 \text{ for non-leaf nodes}$$

$$\text{Initially, } A[i] \leq B[i]$$



S1

Q 1

Q 2

Q 3

Input Format

First argument is an integer array A of size N where A[i] denotes the number of fruits on ith node.

Second argument is an integer array B of size N where B[i] denotes the capacity of ith node.

Third argument is a 2D array C of size (N-1) x 2 denotes edges in the tree. There is an edge between nodes C[i][0] and C[i][1].

Output Format

Return an integer denoting the minimum number of shakes you have to perform such that all the fruits are on the ground.

Example Input

Input 1:

```
A = [0, 0, 0, 1, 1, 2]
```

```
B = [1, 1, 1, 1, 1, 2]
```

```
C = [
```

```
    [1, 2]
```

```
    [1, 3]
```

```
    [2, 5]
```

```
    [2, 6]
```

```
    [3, 4]
```

```
]
```

Input 2:

```
A = [0, 0, 5, 5]
```

```
B = [10, 3, 10, 10]
```

```
C = [
```

```
    [1, 2]
```

```
    [2, 3]
```

```
    [2, 4]
```

```
]
```



S1 ✓

Q 1

Q 2

Q 3

Input 1:

```
A = [0, 0, 0, 1, 1, 2]
B = [1, 1, 1, 1, 1, 2]
C = [
    [1, 2]
    [1, 3]
    [2, 5]
    [2, 6]
    [3, 4]
]
```

Input 2:

```
A = [0, 0, 5, 5]
B = [10, 3, 10, 10]
C = [
    [1, 2]
    [2, 3]
    [2, 4]
]
```

Example Output

Output 1:

4

Output 2:

9

Example Explanation

4

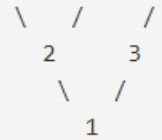
Output 2:

9

Example Explanation

Explanation 1:

Given Tree: 6(2) 5(1) 4(1)

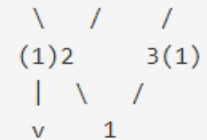


Ground: _____

Leaf nodes are (6, 5, 4).

Now after one shake:

6(1) 5(0) 4(0)

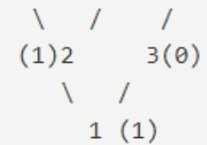


Ground: ____1____

Capacity of node 2 is 1, so the excess fruit i.e. will fall on the ground as shown.

After second shake:

6(0) 5(0) 4(0)



Ground: ____2____

This time there will be excess(i.e 2) fruit at node 1. So one fruit will fall to ground.



S1

Q 1

Q 2

Q 3

Now after one shake:

```

      6(1) 5(0) 4(0)
        \ / /
      (1)2   3(1)
        | \ /
         v  1

```

Ground: ____1____

Capacity of node 2 is 1, so the excess fruit i.e. will fall on the ground as shown.

After second shake:

```

      6(0) 5(0) 4(0)
        \ / /
      (1)2   3(0)
        | \ /
         v  1 (1)

```

Ground: ____2____

This time there will be excess(i.e 2) fruit at node 1. So one fruit will fall to ground.

Now, 2 more shakes required to fall all 4 fruits on ground. So total shakes will be 4.

Explanation 2:

Given Tree: 4 3

```

      \ /
      2
      |
      1

```

Ground: ____

Leaf Nodes are 4 and 3. It will require 9 shakes such that all fruits fall on ground.



You only need to implement the given function. Do not read input; instead use the arguments to the function. Do not print the output; instead return values as specified. Still have a question? Check out Sample Codes for more details.





S1

Q 1

Q 2

Q 3

Explanation 2:

Given Tree: 4 3

\ /

2

|

1

Ground: _____

Leaf Nodes are 4 and 3. It will require 9 shakes such that all fruits fall on ground.



You only need to implement the given function. Do not read input; instead use the arguments to the function. Do not print the output; instead return values as specified. Still have a question? Check out Sample Codes for more details.



See Expected Output

HINTS



Hint 1

Editor Mode - Normal



C++17 (gcc-9.2)

```
1 int Solution::solve(vector<int> &A, vector<int> &B, vector<vector<int> > &C) {  
2 }  
3
```




S1



Q 1

Q 2

Q 3

S1 Default Section



Q1 >



Q2 >



Q3



Damaged Roads

[Flag Question](#)

You are the Prime Minister of a country and once you went for a world tour.

After 5 years, when you returned to your country, you were shocked to see the condition of the roads between the cities. So, you plan to repair them, but you cannot afford to spend a lot of money.

The country can be represented as a $N \times M$ grid, where **Country(i, j)** is a city.

The cost of repairing a road between **(i, j)** and **(i + 1, j)** is **A[i]**. The cost of repairing a road between **(i, j)** and **(i, j + 1)** is **B[i]**.

Return the **minimum** cost of repairing the roads.

As the cost can be large, return the cost modulo 10^9+7 .

Input Format:

The first argument will be an integer array, A, of size N.
The second argument will be an integer array, B, of size M.

Output Format:

Return an integer representing the minimum possible cost.

Constraints:

$1 \leq N, M \leq 1e5$
 $1 \leq A[i], B[i] \leq 1e3$

Example:

Input 1:



S1 ✓

Q 1

Q 2

Q 3

Return the **minimum** cost of repairing the roads.

As the cost can be large, return the cost modulo 10^9+7 .

Input Format:

The first argument will be an integer array, A, of size N.
The second argument will be an integer array, B, of size M.

Output Format:

Return an integer representing the minimum possible cost.

Constraints:

$1 \leq N, M \leq 1e5$
 $1 \leq A[i], B[i] \leq 1e3$

Example:

Input 1:

A = [1, 1, 1]

B = [1, 1, 2]

Output 1:

16

Input 2:

A = [1, 2, 3]

B = [4, 5, 6]

Output 2:

39



You only need to implement the given function. Do not read input; instead use the arguments to the function. Do not print the output; instead return values as specified. Still have a question? Check out Sample Codes for more details.





S1

Q 1

Q 2

Q 3

Output 2:

39



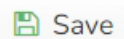
You only need to implement the given function. Do not read input; instead use the arguments to the function. Do not print the output; instead return values as specified. Still have a question? Check out Sample Codes for more details.

[See Expected Output](#)

Editor Mode - Normal

C++17 (gcc-9.2)

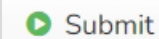
```
1 int Solution::solve(vector<int> &A, vector<int> &B) {  
2 }  
3
```



Save



Reset



Submit



Test



Test with Custom Input



S1

Q 1

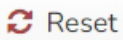
Q 2

Q 3

```
68 int Solution::solve(vector<int> &A, vector<int> &B) {  
69     test t;  
70     return t.findMinCost(A, B);  
71 }  
72
```



Save



Reset



Submit



Test



Test with Custom Input

Compilation

▼ Trivial Case Correctness

Wrong Answer. Your program's output doesn't match the expected output. You can try testing your code with **custom input** and try putting **debug statements** in your code.

Your submission failed for the following input:

```
A : [ 2, 2, 4, 1, 2, 4, 2 ]  
B : [ 3, 3, 4, 5 ]
```

Your function returned the following :

```
71
```

The expected returned value :

```
96
```

Run as custom input

💡 Make sure you are not printing values inside your function

i

S1

Q 1

Q 2

Q 3

```
79 int Solution::solve(vector<int> &A, vector<int> &B) {
80     test t;
81     return t.findMinCost(A, B);
82 }
83
```

SaveReset

SubmitTestTest with Custom Input

Compilation

Trivial Case Correctness

Wrong Answer. Your program's output doesn't match the expected output. You can try testing your code with **custom input** and try putting **debug statements** in your code.

Your submission failed for the following input:

```
A : [ 1, 3 ]
B : [ 5, 4 ]
```

Your function returned the following :

```
10
```

The expected returned value :

```
21
```

Run as custom input

Make sure you are not printing values inside your function



S1

Q 1

Q 2

Q 3

S1 Default Section

Q1 > Q2 > Q3

Omega Primes

[Flag Question](#)

Problem Description

Carl is bored of playing with ordinary prime numbers. Thus, he comes up with some special numbers called Omega Primes.

A number **X** is called Omega Prime, if there exists no perfect square **Y** ($Y > 1$) such that Y divides X.

For example, 6 is an Omega Prime because there is no perfect square except 1 that divides 6.

On the other hand, 12 is not an Omega Prime as 4 (which is a perfect square) is a divisor of 12.

Carl decides to play a bit more with Omega Primes. He has an array **A** of integers. Carl wants to find the number of different subsets such that the product of elements for each subset, results in an Omega Prime. Help Carl find this number.

Since this number can be large, output the answer modulo $10^9 + 7$.

Problem Constraints

$1 \leq \text{Length of A} \leq 2 * 10^4$

$1 \leq A[i] \leq 30$

Input Format

The first argument is the integer array A.

Output Format



S1



Q 1

Q 2

Q 3

Problem Constraints $1 \leq \text{Length of } A \leq 2 * 10^4$
 $1 \leq A[i] \leq 30$ **Input Format**

The first argument is the integer array A.

Output Format

Return an integer denoting the number of different desired subsets modulo $10^9 + 7$.

Example Input

Input 1:

A = [2, 4, 3]

Input 2:

A = [2, 2, 2]

Example Output

Output 1:

3



S1



Q 1

Q 2

Q 3

Input 1:

```
A = [2, 4, 3]
```

Input 2:

```
A = [2, 2, 2]
```

Example Output

Output 1:

```
3
```

Output 2:

```
3
```

Example Explanation

Explanation 1:

```
The different subsets are [0, 2], [0], [2]. (the numbers denote the indices of the array elements)
```

Explanation 2:

```
The different subsets are [0], [1], [2].
```

```
Note, the Omega Primes generated are same but the subset used to generate them are different.
```



S1

Q 1

Q 2

Q 3

Example Explanation

Explanation 1:

The different subsets are [0, 2], [0], [2]. (the numbers denote the indices of the array elements)

Explanation 2:

The different subsets are [0], [1], [2].

Note, the Omega Primes generated are same but the subset used to generate them are different.



You only need to implement the given function. Do not read input; instead use the arguments to the function. Do not print the output; instead return values as specified. Still have a question? Check out Sample Codes for more details.

[See Expected Output](#)

HINTS



Hint 1

Editor Mode - Normal



C++17 (gcc-9.2)

```
1 int Solution::findOmegas(vector<int> &A) {  
2 }  
3
```